

RISC-V Instruction Set Architecture

Outline

- Introduction
- MIPS review
- Register convention
- R-format
- I-format
- S-format
- B-format
- U-format
- J-format
- Summary
- Reference

2017 Turing Award

April 17, 2018

On Monday June 4, 2018, 2017 A.M. Turing Award Winners John L. Hennessy and David A. Patterson will deliver the Turing Lecture at the 45th International Symposium on Computer Architecture ([ISCA](#)) in Los Angeles.

The celebrated computer architects have penned this short article ([published on the ISCA website](#)), previewing their talk.

**“A New Golden Age for
Computer Architecture:
Domain-Specific Hardware/Software Co-Design, Enhanced
Security, Open Instruction Sets, and Agile Chip
Development”**

by John L. Hennessy and David A. Patterson



CISC vs. RISC Today (as of 2016)

- PC Era
- Hardware translates x86 instructions into internal RISC instructions
- Then use any RISC technique inside MPU
- > 350M / year !
- x86 ISA eventually dominates servers as well as desktops
- PostPC Era: Client/Cloud
- IP in SoC vs. MPU
- Value die area, energy as much as performance
- > 18B / year in 2016!
- 98% RISC Processors:
 - 15.0B ARM
 - 2.0B Tensilica
 - 1.5B ARC (Synopsys)
 - 0.8B MIPS

Consensus on ISAs Today

- Not CISC: no new commercial CISC ISAs in 30+ years
- Not VLIW: Despite several attempts, VLIW has failed in general-purpose computing arena
 - Complex VLIW architectures close to in-order superscalar in complexity, no real advantage on large complex apps
 - Although VLIWs successful in embedded DSP market (Simpler VLIWs, more constrained, friendlier code)
- RISC! Widespread agreement (still) that RISC principles are best for general purpose ISA

Golden Age in Microprocessor Design

- 40 years stunning progress ($\approx 10^6x$) in microprocessor design
- Three architectural innovations ($\sim 1000x$)
 - Width: 8 $\rightarrow$ 16 $\rightarrow$ 32 $\rightarrow$ 64 bit ($\sim 8x$)
 - Instruction level parallelism:
 - 4-10 CPI to 4+ IPC ($\sim 10-20x$)
 - Multicore: 1 processor to 16 cores ($\sim 16x$)
- Clock rate: 3 to 4000 MHz ($\sim 1000x$)
 - Through technology & architecture
- Made possible by IC technology:
 - **Moore's Law**: growth in transistor count (2X every 1.5 years)
 - **Dennard Scaling**: power/transistor shrinks at same rate as transistors are added (constant per mm^2 of silicon)

However, ...

■ Technology

- End of Dennard scaling: power becomes the key constraint
- Slowdown (retirement) of Moore's Law: transistors cost

■ Architectural

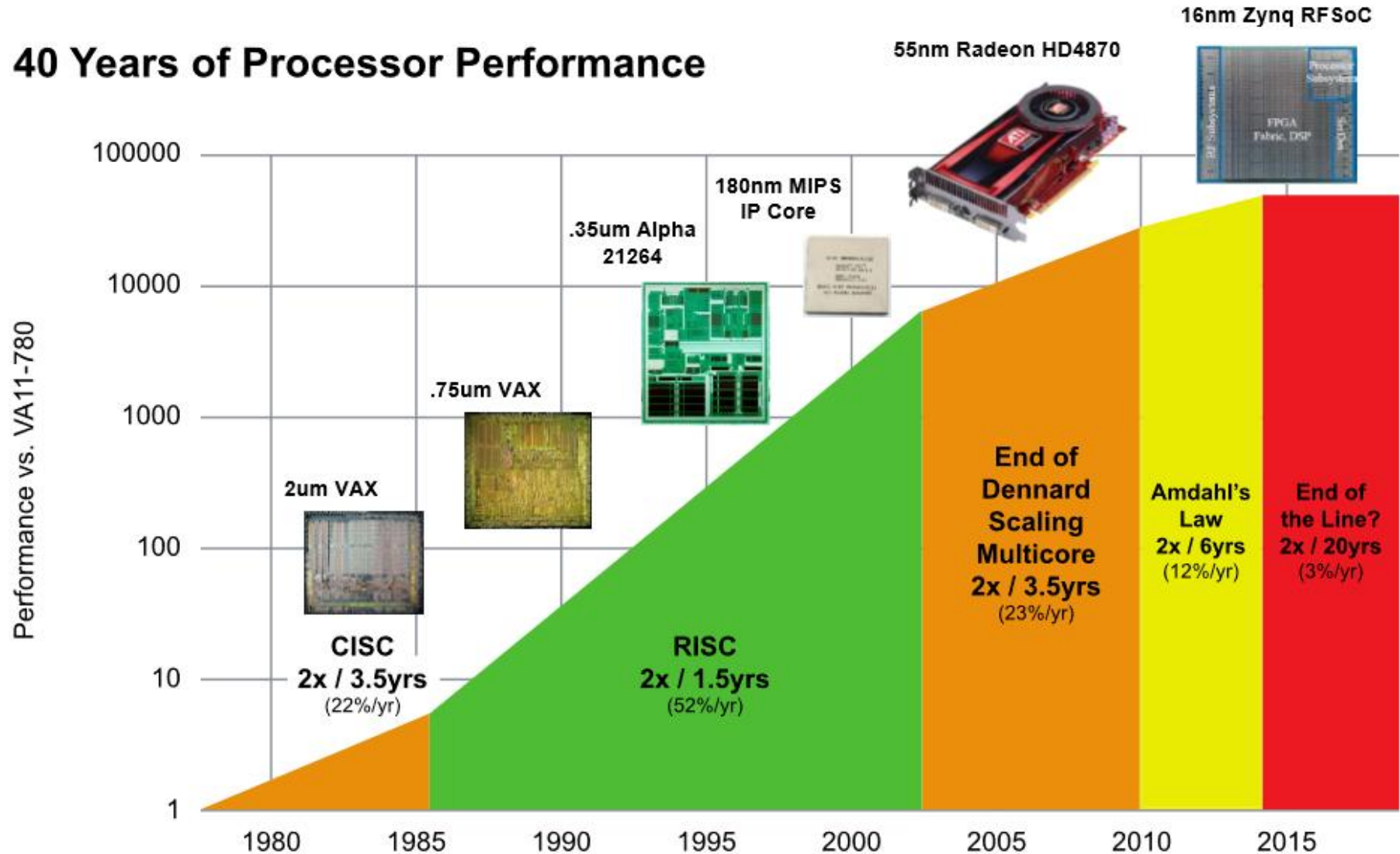
- Limitation and inefficiencies in exploiting instruction level parallelism end the uniprocessor era in 2004
- Amdahl's Law and its implications end "easy" multicore era

■ Products

- PC/Server $\Rightarrow$ Client/Cloud

The End of Performance Growth?

40 Years of Processor Performance



Based on SPECintCPU. Source: John Hennessy and David Patterson, Computer Architecture: A Quantitative Approach, 6/e. 2018

Domain-Specific Architecture

- End of Dennard scaling and Moore's Law $\Rightarrow$ improving cost-energy-performance requires innovation in architecture
- Potential upside of domain-specific architectures and synergy with domain-specific programming languages
 - Google TensorFlow and TPU (30~80x perf./Watt over GPU & CPU)
- Innovators can demonstrate ideas with FPGAs and custom chips instead of software simulators
- Reduced software non-recurring engineering costs with open instruction set (e.g., RISC-V)
- Reduced hardware NRE with open IP blocks
- Better productivity via agile hardware development & tools (e.g., Chisel)
*趨勢: 眾星拱月, 許多加速器 surrounding CPU, for domain-specific usages
↑ RISC-V enables.*

What is RISC-V ?

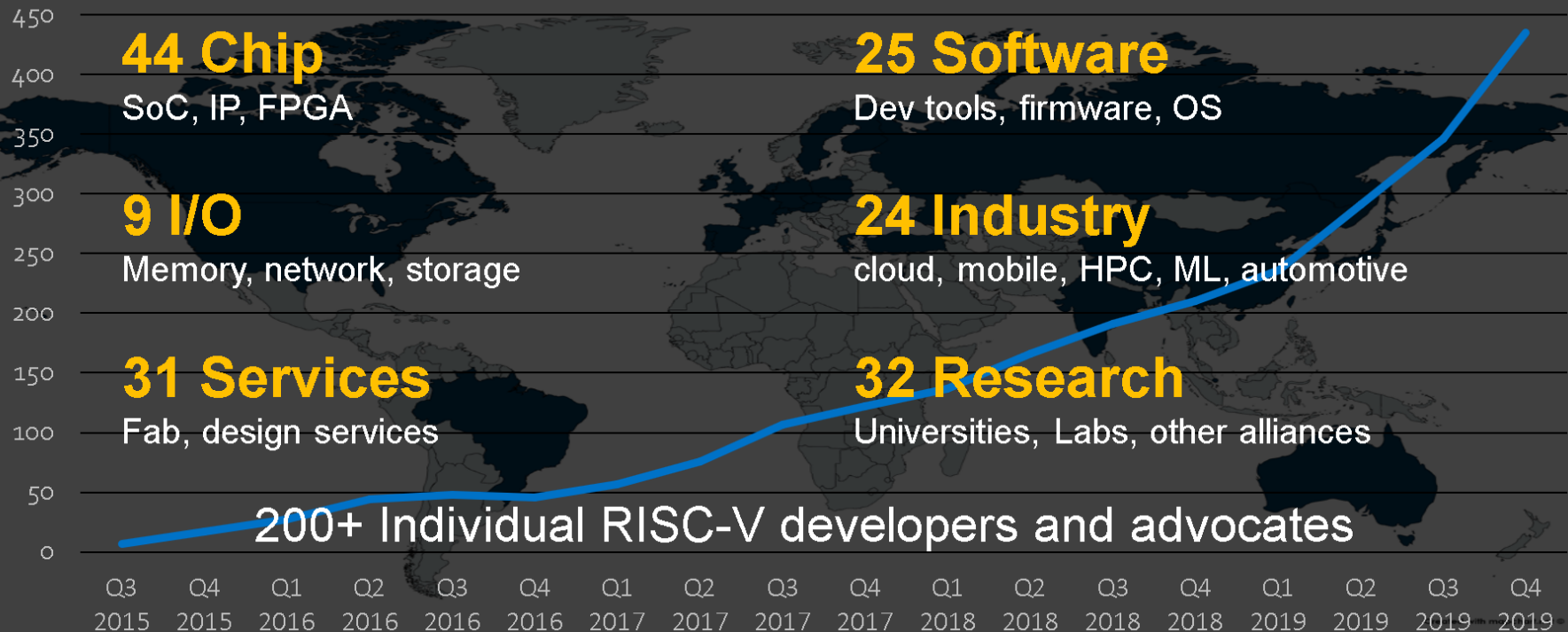


- RISC-V(pronounced "risk-five") is an open source implementation of a reduced instruction set computing (RISC) based instruction set architecture (ISA)
- Most ISAs are commercially protected by patents, preventing practical efforts to reproduce the computer systems. In contrast, RISC-V is open, permitting any person or group to construct compatible computers, and use associated software.
- The project was originated in 2010 by researchers in the Computer Science Division at UC Berkeley, but it now has a large number of contributors. As of 2019 version 2.2 of the userspace ISA is fixed.

- A completely open ISA that is freely available to academia and industry.
- A real ISA suitable for direct native hardware implementation, not just simulation or binary translation.
- An ISA that avoids "over-architecting" for a particular microarchitecture style, but which allows efficient implementation in any of these.
- An ISA separated into a small base integer ISA, usable by itself as a base for customized accelerators or for educational purpose, and optional standard extensions, to support general- purpose software development.



More than 435 RISC-V Members (as of Dec. 2019)



RISC-V VS. OTHERS



	X86 , ARM	RISC-V
Manual pages	Thousands	Less than 300
Instructions	Many and branches incompatibility	Basic instruction less than 50
Modularity	NO	YES
Expandable	NO	YES
Hardware implement	Complicate	Simple
Fee	Very expensive	Free
Development	Mature	Growing up very fast
Risk	Uncontrollable, low flexibility, high cost	Patent

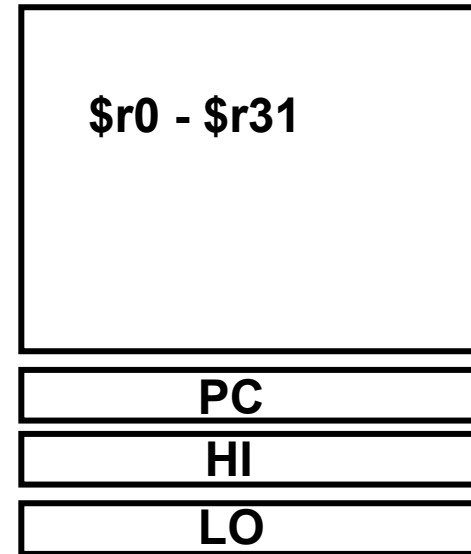
} ARM: 賣固定的IP

} 模組化, 可自由搭配

■ Instruction categories:

- Load/Store
- Computational
- Jump and Branch
- Floating Point
- Memory Management
- Special

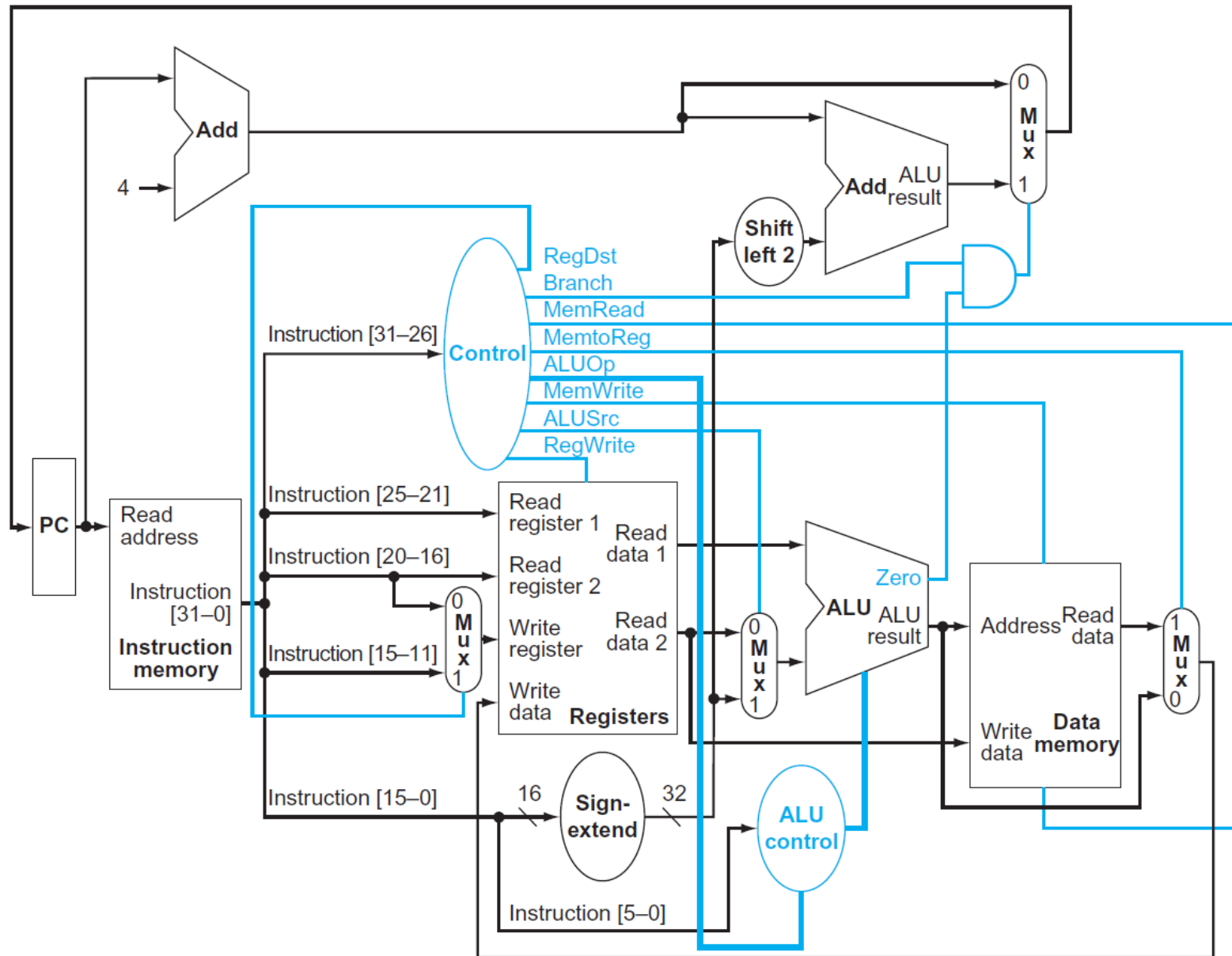
Registers



3 Instruction Formats: all 32 bits wide

OP	\$rs	\$rt	\$rd	sa	func	R-format
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	
OP	\$rs	\$rt	immediate			I-format
6 bits	5 bits	5 bits	16 bits			
OP	jump target					J-format
6 bits	26 bits					

Single-Cycle MIPS Implementation



RISC-V ISA Base and Extension

- RISC-V has a modular design, consisting of alternative base parts, with added optional extensions.
- The standard extensions are specified to work with all of the standard bases, and with each other without conflict.

Base ISA

Name	Description	Version
RV32I	Base Integer Instruction Set 32-bit	2.0
RV32E	Base Integer Instruction Set (embedded) 32-bit, 16 registers	1.9
RV64I	Base Integer Instruction Set 64-bit	2.0
RV128I	Base Integer Instruction Set 128-bit	1.7

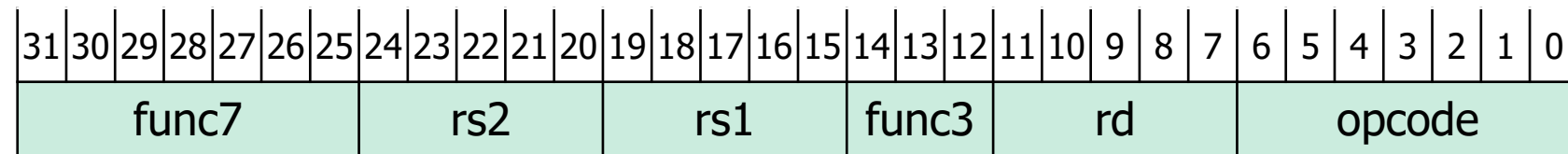
RISC-V ISA Base and Extension (Cont.)

Extensions ISA

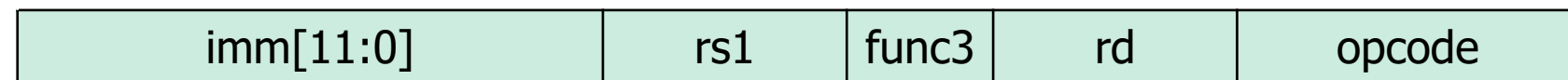
Name	Description	Version
M	Standard Extension for Integer Multiplication and Division	2.0
A	Standard Extension for Atomic Instructions	2.0
F	Standard Extension for Single-Precision Floating-Point	2.0
D	Standard Extension for Double-Precision Floating-Point	2.0
G	Shorthand for the base and above extensions	N/A
Q	Standard Extension for Quad-Precision Floating-Point	2.0
L	Standard Extension for Decimal Floating-Point	0.0
C	Standard Extension for Compressed Instructions	2.0
B	Standard Extension for Bit Manipulation	0.90
V	Standard Extension for Vector Operations	0.2
N	Standard Extension for User-Level Interrupts	1.1

不需要就不拿某個 subset → 節省晶片面積

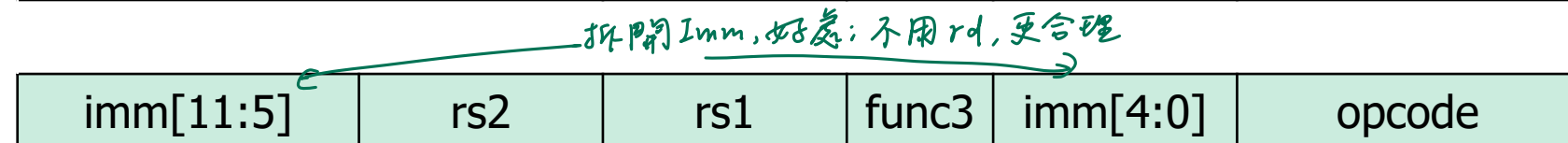
RISC-V Instruction Formats



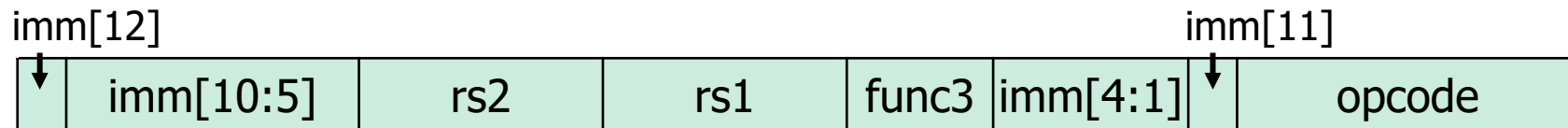
R-format



I-format



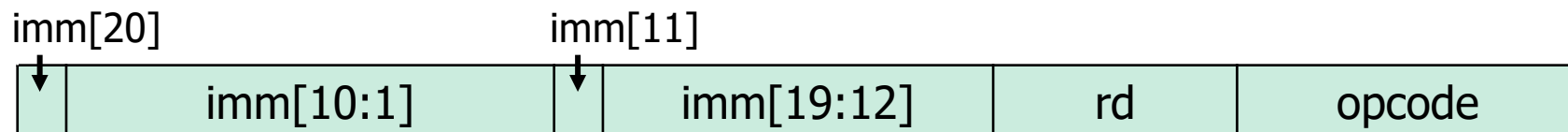
S-format



B-format



U-format

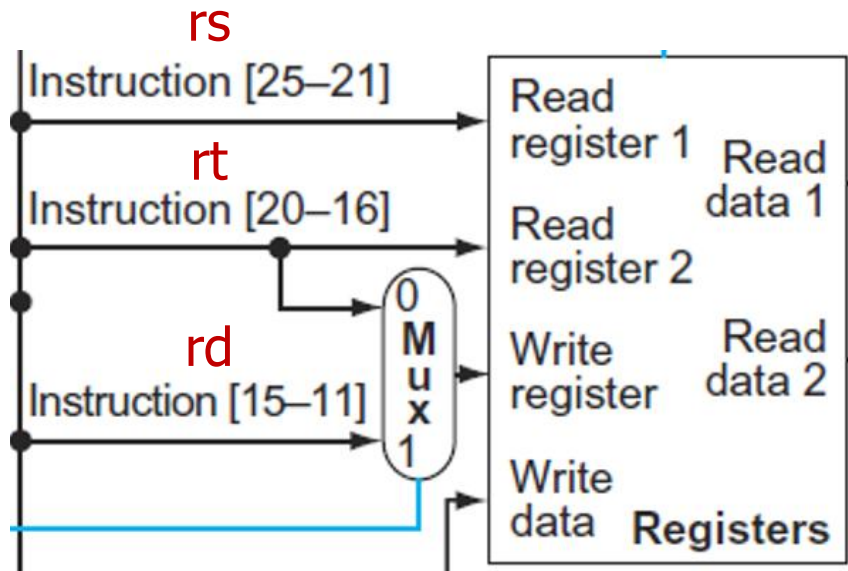


J-format

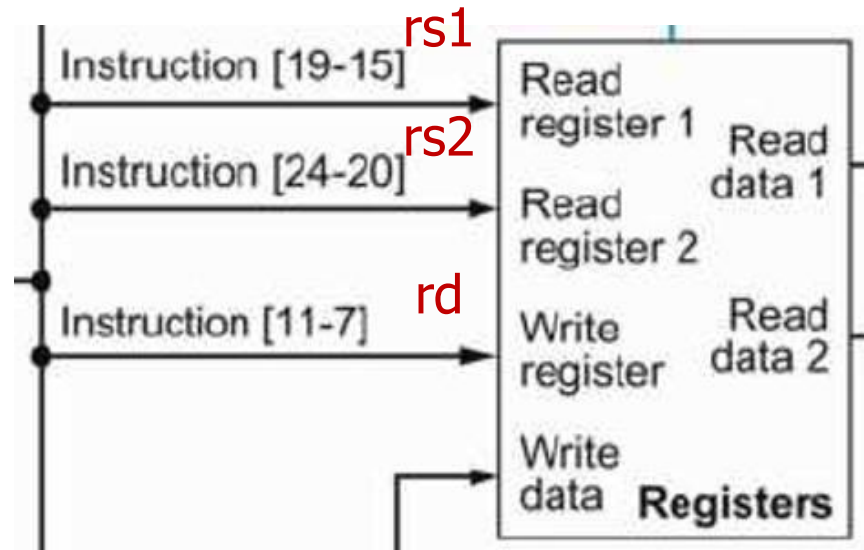
7-bit

Regularity of Register Fields

■ MIPS



■ RISC-V



永遠寫 rd, 省一個 mux!

RISC-V Register Convention



Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	-
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	-
x4	tp	Thread pointer	-
x5	t0	Temporary/alternate link register	Caller
x6-7	t1-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

MIPS and RISC-V Registers

Name	Number
\$zero	0
\$v0 - \$v1	2 - 3
\$a0 - \$a3	4 - 7
\$t0 - \$t7	8 - 15
\$s0 - \$s7	16 - 23
\$t8 - \$t9	24 - 25
\$gp	28
\$sp	29
\$fp	30
\$ra	31
\$hi	x
\$lo	x

Name	Func
x0	zero
x1	ra
x2	sp
x3	gp
x4	tp
x5	t0
x6-7	t1-2
x8	s0/fp
x9	s1
x10-11	a0-1
x12-17	a2-7
x18-27	s2-11
x28-31	t3-6

R-Format

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- opcode (operation code): partially specifies what instruction it is, and R-format opcode is fixed.
- rd (destination register): specifies register which will receive result of the operation.
- funct3 (3-bit function code): additional opcode.
- rs1 (1st source register): specifies the first register operand.
- rs2 (2nd source register): specifies the second register operand.
- funct7 (7-bit function code): additional opcode.

R-Format

- In R-format, all operations read the rs1 and rs2 registers as source operands and write the result into register rd.
- Use funct7 and funct3 fields to distinct different R-format instructions.
- In the following slides, we discuss R-format instructions: ADD and SLT.
 - ADD: $rd = rs1 + rs2$.
 - SLT (set on less than): if $rs1 < rs2$, then $rd = 1$; otherwise $rd = 0$.

R-Format Example

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

add x9, x20, x21

0	21	20	0	9	51
---	----	----	---	---	----

0000000	10101	10100	000	01001	0110011
---------	-------	-------	-----	-------	---------

0000 0001 0101 1010 0000 0100 1011 0011₂ = 015A04B3₁₆

R-Format Example

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

slt x20, x25, x26

0	26	25	2	20	51
---	----	----	---	----	----

0000000	11010	11001	010	10100	0110011
---------	-------	-------	-----	-------	---------

0000 0001 1010 1100 1010 1010 0011 0011₂ = 01ACAA33₁₆

RV32I R-Format Instructions

inst	funct7	rs2	rs1	funct3	rd	opcode
ADD	0000000	rs2	rs1	000	rd	0110011
SUB	0100000	rs2	rs1	000	rd	0110011
SLL	0000000	rs2	rs1	001	rd	0110011
SLT	0000000	rs2	rs1	010	rd	0110011
SLTU	0000000	rs2	rs1	011	rd	0110011
XOR	0000000	rs2	rs1	100	rd	0110011
SRL	0000000	rs2	rs1	101	rd	0110011
SRA	0100000	rs2	rs1	101	rd	0110011
OR	0000000	rs2	rs1	110	rd	0110011
AND	0000000	rs2	rs1	111	rd	0110011

I-Format

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

- opcode: partially specifies what instruction it is
- rd: specifies register which will receive result of the operation
- funct3: additional opcode
- rs1: specifies the only register operand, i.e., source or base address register
- immediate: constant operand, or offset added to base address
 - 2's complement, sign extended

I-Format

- I-format is usually used for immediate arithmetic and load instructions.
- Distinct these two types with different opcodes.
 - Immediate arithmetic opcode: 0010011
 - Load opcode: 0000011
- Use funct3 and opcode fields to distinct different I-format instructions.
- In the following slides, we discuss I-format instructions: ADDI, SLTIU, and LW
 - ADDI: $rd = rs1 + \text{sign_extended}(\text{imm})$.
 - SLTIU: if $rs1 < \text{zero_extended}(\text{imm})$, then $rd = 1$; otherwise $rd = 0$.
 - LW: $rd = \text{MEM}[rs1 + \text{sign_extended}(\text{imm})]$.

I-Format Example

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

addi x15, x1, -50

-50	1	0	15	19
-----	---	---	----	----

111111001110	00001	000	01111	0010011
--------------	-------	-----	-------	---------

$1111\ 1100\ 1110\ 0000\ 1000\ 0111\ 1001\ 0011_2 = \text{FCE08793}_{16}$

I-Format Example

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

sltiu x19, x18, 33

33	18	3	19	19
----	----	---	----	----

000000100001	10010	011	10011	0010011
--------------	-------	-----	-------	---------

0000 0010 0001 1001 0011 1001 1001 0011₂ = 02193993₁₆

I-Format Example

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

ld x9, 240(x10)

240	10	3	9	3
-----	----	---	---	---

000011110000	01010	011	01001	0000011
--------------	-------	-----	-------	---------

$$0000\ 1111\ 0000\ 0101\ 0011\ 0100\ 1000\ 0011_2 = 0F053483_{16}$$

All RV32 Load Instructions

Immediate	rs1	func3	rd	opcode	type
Imm[11:0]	rs1	000	rd	0000011	LB
Imm[11:0]	rs1	001	rd	0000011	LH
Imm[11:0]	rs1	010	rd	0000011	LW
Imm[11:0]	rs1	100	rd	0000011	LBU
Imm[11:0]	rs1	101	rd	0000011	LHU



func3 field encodes size and signedness of load data

- LBU is "load byte unsigned"
- LH is "load half", which loads 16 bits (2 bytes) and sign-extends to fill destination 32-bit register
- LHU is "load half unsigned", which zero-extends 16 bits to fill destination 32-bit register
- There is no LWU in RV32, since no sign/zero extension is required when copying 32 bits from a memory to register

RV32I I-Format Instructions

inst	imm[11:0]		rs1	funct3	rd	opcode
ADDI	imm[11:0]		rs1	000	rd	0010011
SLTI	imm[11:0]		rs1	010	rd	0010011
SLTIU	imm[11:0]		rs1	011	rd	0010011
XORI	imm[11:0]		rs1	100	rd	0010011
ORI	imm[11:0]		rs1	110	rd	0010011
ANDI	imm[11:0]		rs1	111	rd	0010011
SLLI	0000000	shamt	rs1	001	rd	0010011
SRLI	0000000	shamt	rs1	101	rd	0010011
SRAI	0100000	shamt	rs1	101	rd	0010011
LB	imm[11:0]		rs1	000	rd	0000011
LH	imm[11:0]		rs1	001	rd	0000011
LW	imm[11:0]		rs1	010	rd	0000011
LBU	imm[11:0]		rs1	100	rd	0000011
LHU	imm[11:0]		rs1	101	rd	0000011

RISC-V Logical Instructions

<i>Instruction</i>	<i>Example</i>	<i>Meaning</i>	<i>Comment</i>
and	and x9,x10,x11	$x9 = x10 \& x11$	3 reg. operands; Logical AND
or	or x9,x10,x11	$x9 = x10 x11$	3 reg. operands; Logical OR
xor	xor x9,x10,x11	$x9 = x10 \wedge x11$	3 reg. operands; Logical XOR
and immediate	andi x9,x10,4	$x9 = x10 \& 4$	Logical AND reg, sign extended
or immediate	ori x9,x10,4	$x9 = x10 4$	Logical OR reg, sign extended
xor immediate	xori x9,x10,4	$x9 = x10 \wedge 4$	Logical XOR reg, sign extended
shift left	sll x9,x10,x11	$x9 = x10 \ll x11$	Shift left by constant in reg
shift left imm.	slli x9,x10,4	$x9 = x10 \ll 4$	Shift left by constant
shift right	srl x9,x10,x11	$x9 = x10 \gg x11$	Shift right by constant in reg
shift right imm.	slli x9,x10,4	$x9 = x10 \gg 4$	Shift right by constant
shift right arith.	sra x9,x10,x11	$x9 = x10 \gg x11$	Shift right (sign extend)
shift right arith. imm.	srai x9,x10,4	$x9 = x10 \gg 4$	Shift right (sign extend)

Bitwise Operations

- View contents of register as 32 bits rather than as a single 32-bit number.
- we may want to access individual bits rather than the whole.
- RV32I have R-format and I-format bitwise operation instructions.
- In the following slides, we discuss bitwise operation instructions: OR and ANDI.
 - OR: $rd = rs1 \mid rs2$.
 - ANDI: $rd = rs1 \& \text{sign_extended}(\text{imm.})$.

Bitwise Operation: OR

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

or x17, x13, x15

0	15	13	6	17	51
---	----	----	---	----	----

0000000	01111	01101	110	10001	0110011
---------	-------	-------	-----	-------	---------

0000 0000 1111 0110 1110 1000 1011 0011₂ = 00F6E8B3₁₆

x13 0001 0010 0011 0110 0001 1000 0000 0111₂

x15 1010 1010 1001 1100 0110 1010 0101 0011₂

x17 1011 1010 1011 1110 0111 1010 0101 0111₂

Bitwise Operation: ANDI



immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

andi x15, x18, 3102

3102	18	7	15	19
------	----	---	----	----

110000011110	10010	111	01111	0010011
--------------	-------	-----	-------	---------

0000 0001 1110 1001 0111 0111 1001 0011₂ = 01E97793₁₆

x18	0000 0110 0000 0110 1111 1111 1111 1100 ₂
-----	--

x15	0000 0110 0000 0110 1111 1100 0001 1100 ₂
-----	--

Shift Operations

- RV32I have R-format and I-format shift operation instructions.
- Only use imm[4:0] to shift, because 5 bits ($2^5 = 32$) are enough for 32-bit data.
- Two shift right operations: shift right logical and shift right arithmetic.
 - srl: fills MSB with zero.
 - sra: fills MSB by sign extension.
- In the following slides, we discuss bitwise operation instructions: SRA and SLLI.
 - SRA: $rd = \text{sign_extended}(rs1 \gg rs2[4:0])$.
 - SLLI: $rd = rs1 \ll \text{imm}[4:0]$.

Shift Operation: SRA

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

sra x19, x23, x18 Assume that x18 is 8

32	18	23	5	19	51
----	----	----	---	----	----

0100000	10010	10111	101	10011	0110011
---------	-------	-------	-----	-------	---------

0100 0001 0010 1011 1101 1001 1011 0011₂ = 412BD9B3₁₆

x23 1000 1111 0111 0000 1100 1101 1100 0001₂

x19 1111 1111 1000 1111 0111 0000 1100 1101₂

Shift Operation: SLLI

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

slli x19, x21, 3

3	21	1	19	3
---	----	---	----	---

0000000000011	10101	001	10011	0000011
---------------	-------	-----	-------	---------

0000 0000 0011 1010 1001 1001 1000 0011₂ = 003A9983₁₆

x21

1110	0010	0111	1010	1011	1101	0101	0010 ₂
------	------	------	------	------	------	------	-------------------

x19

0001	0011	1101	0101	1110	1010	1001	0000 ₂
------	------	------	------	------	------	------	-------------------

S-Format

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- opcode: partially specifies what instruction it is
- funct3: additional opcode
- rs1: base address register number
- rs2: source operand register number
- immediate: offset added to base address
 - Split so that rs1 and rs2 fields always in the same place
- Used for store instructions.
- In the following slide, we discuss S-format instruction: SW.
 - SW: $\text{MEM}[\text{rs1} + \text{sign_extended}(\text{imm})] = \text{rs2}.$

S-Format Example

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

sw x14, 8(x2)

0	14	2	2	8	35
---	----	---	---	---	----

0000000	01110	00010	010	01000	0100011
---------	-------	-------	-----	-------	---------

0000 0000 1110 0001 0010 0100 0010 0011₂ = E12423₁₆

RV32I Store Instruction



- RV32I support 3 different store instructions.
- Use funct3 field to distinct different load instructions.
- SW: store a word (32bits).
- SH: store a half word (16bits).
- SB: store a byte (8bits).

Inst	SB	SH	SW
Funct3	000	001	010

inst	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[11:5]	rs2	rs1	000	imm[4:0]	0100011
SH	imm[11:5]	rs2	rs1	001	imm[4:0]	0100011
SW	imm[11:5]	rs2	rs1	010	imm[4:0]	0100011

B-Format

imm[12]	imm[10:5]	rs2	rs1	func3	Imm[4:1]	Imm[11]	opcode
1 bit	6 bit	5 bit	5 bit	3 bit	4 bit	1 bit	7 bit

- Opcode, two registers (rs1, rs2), and target address (imm)
- Unusual encoding for imm, but simplifies datapath
- Immediate is only 12 bits, but PC is 64-bit
 - immediate cannot specify entire address
- Loops are generally small: < 50 instructions
 - Though we want to branch to anywhere in memory, a single branch only need to change PC by a small amount
- How to use PC-relative addressing
 - Target address = $PC + imm \times 2$
 - -4096 to 4094, in multiples of 2 (even address)

B-Format Example

imm[12]	imm[10:5]	rs2	rs1	func3	Imm[4:1]	Imm[11]	opcode
1 bit	6 bit	5 bit	5 bit	3 bit	4 bit	1 bit	7 bit

bne x10, x11, 2000

0	62	11	10	1	8	0	103
---	----	----	----	---	---	---	-----

0	111110	01011	01010	001	1000	0	1100111
---	--------	-------	-------	-----	------	---	---------



0111 1100 1011 0101 0001 1000 0110 0111₂ = 7CB51867₁₆

Discard the 0-bit and sign extension 11-bit to 12-bit

RV32I Conditional Branch

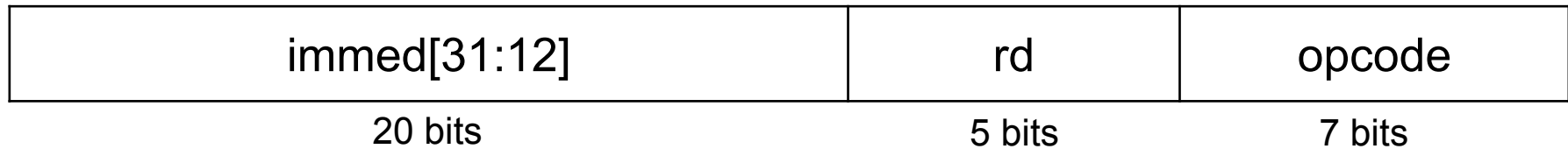
- RV32I support 6 different conditional branches.
- Use funct3 field to distinct different instructions.
- BEQ: branch on equal $\rightarrow rs1 == rs2$.
- BNE: branch on not equal $\rightarrow rs1 != rs2$.
- BLT: branch on less than $\rightarrow rs1 < rs2$.
- BGE: branch on greater or equal $\rightarrow rs1 \geq rs2$.
- BLTU: equal to BLT but use unsigned immediate.
- BGEU: equal to BGE but use unsigned immediate.

Inst	BEQ	BNE	BLT	BGE	BLTU	BGEU
Funct3	000	001	100	101	110	111

RV32I B-Format Instructions

inst	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode
BEQ	imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011
BNE	imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011
BLT	imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011
BGE	imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011
BLTU	imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011
BGEU	imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011

U-Format



- Most constants are small
 - 12-bit immediate is sufficient
- For the occasional 32-bit constant, we need a new instruction format for dealing with the rest of the 20 bits.
- In the following slides, we discuss U-format instructions: LUI and AUIPC.
 - LUI: load upper immediate
 - AUIPC: add upper immediate with pc

U-Format Example

immed[31:12]	rd	opcode
20 bits	5 bits	7 bits

lui x19, 976

976	19	55
-----	----	----

00000000001111010000	10011	0110111
----------------------	-------	---------

0000 0000 0011 1101 0000 1001 1011 0111₂ = 3D09B7₁₆

imm = 976 = 0x003D0

x19	0000 0000 0011 1101 0000	0000 0000 0000
-----	--------------------------	----------------

Copy immediate to rd's higher 20 bits and set the lower 12 bits to zeros.

U-Format Example

immed[31:12]	rd	opcode
20 bits	5 bits	7 bits

auipc x10, 338

338	10	23
-----	----	----

000000000000101010010	01010	0010111
-----------------------	-------	---------

$0000\ 0000\ 0001\ 0101\ 0010\ 0101\ 0001\ 0111_2 = 152517_{16}$

AUIPC forms a 32-bit offset from the 20-bit U-immediate, filling in the lower 12 bits with zeros, adds this offset to the pc, and places the result in register rd.

J-Format

imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1 bit	10 bits	1 bit	8 bits	5 bits	7 bits

- jal saves PC+4 in register rd (the return address).
- Set $PC = PC + imm * 2$.
- Can jump $\pm 2^{18}$ 32-bit instructions.
- **Key concepts:**
 - Keep opcode field identical to the other formats for consistency
 - Combine other fields to make room for target address

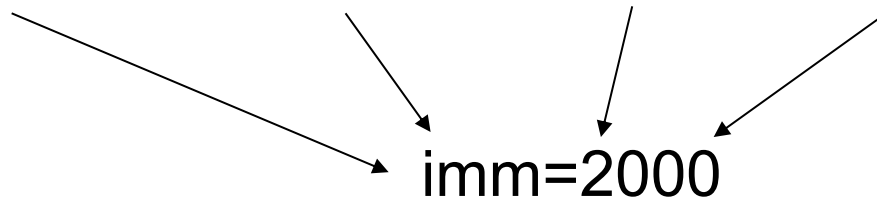
J-Format Example

imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1 bit	10 bits	1 bit	8 bits	5 bits	7 bits

jal x0, 2000

0	1000	0	0	0	111
---	------	---	---	---	-----

0	1111101000	0	00000000	00000	1101111
---	------------	---	----------	-------	---------

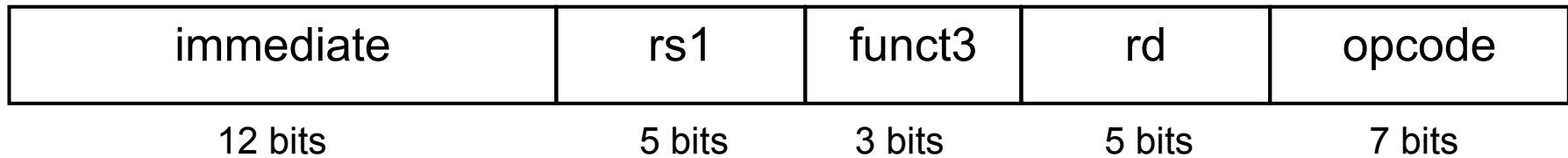


$0000\ 0000\ 0011\ 1101\ 0000\ 1001\ 1011\ 0111_2 = 3D09B7_{16}$

Same as B-Format

Discard the 0-bit and sign extension 19-bit to 20-bit

JALR Instruction



- jalr instruction uses I-Format instead of J-Format.
- Set $rd = PC + 4$.
- Set $PC = rs1 + \text{sign_extended}(\text{imm})$.
- Set PC LSB to 0.
- For long jumps to any 32-bit address, we use a two-instruction sequence
 - lui: load address[31:12] to temp register
 - jalr: add address[11:0] and jump to target

Why is Immediate So Confusing?

Instruction Encoding, inst[31:0]

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

∴ RISC-V 有短指令 (2 bytes)
 ⇒ branch 必须满足 "half word alignment"

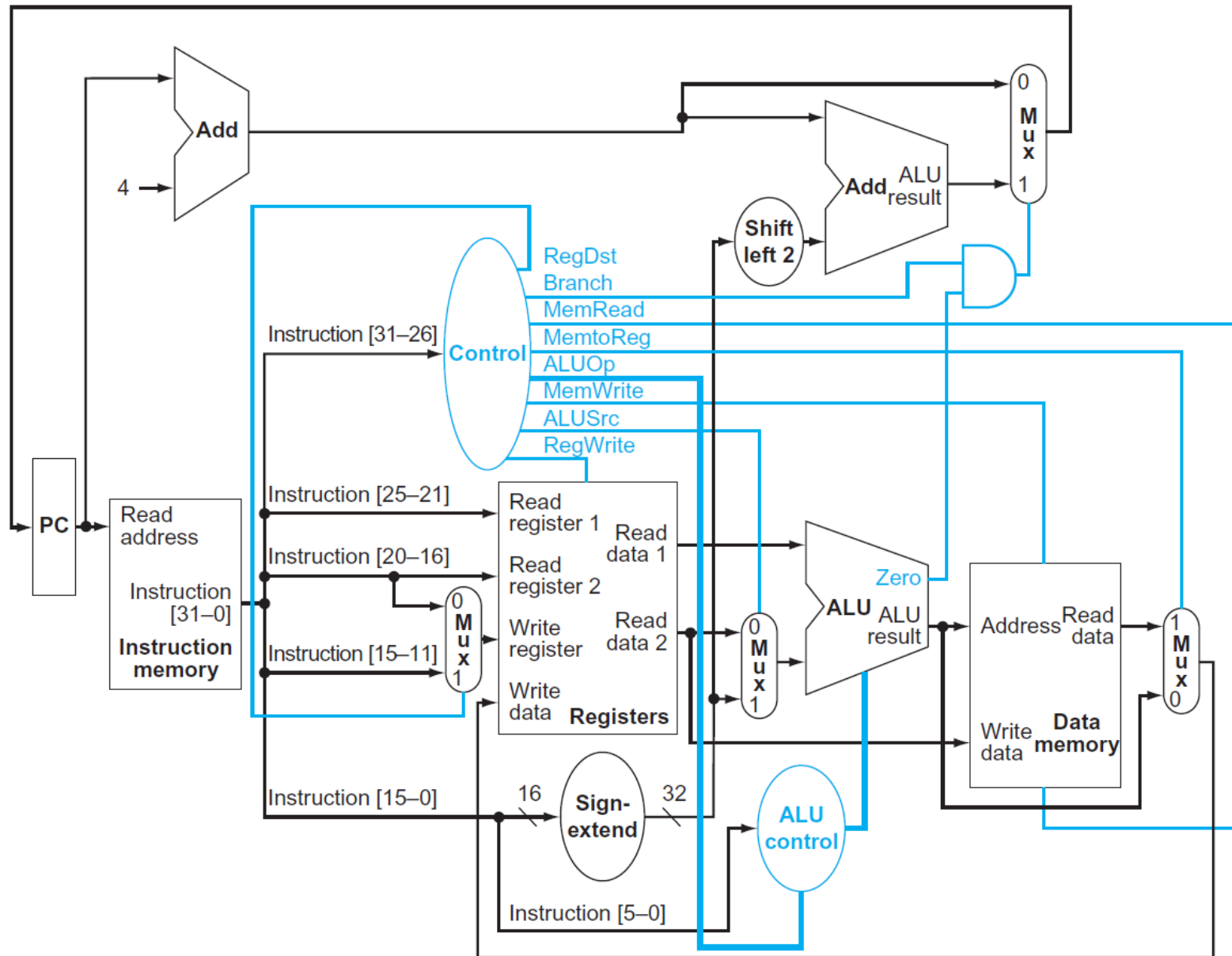
32-bit immediates produced, imm[31:0]

31	30	20	19	12	11	10	5	4	1	0			
— inst[31] —						inst[30:25]		inst[24:21]		inst[20]	I-immediate		
— inst[31] —						inst[30:25]		inst[11:8]		inst[7]	S-immediate		
— inst[31] —						inst[7]	inst[30:25]		inst[11:8]		<u>0</u>	B-immediate	
inst[31]	inst[30:20]				inst[19:12]		— 0 —					U-immediate	
— inst[31] —						inst[19:12]		inst[20]	inst[30:25]		inst[24:21]	<u>0</u>	J-immediate

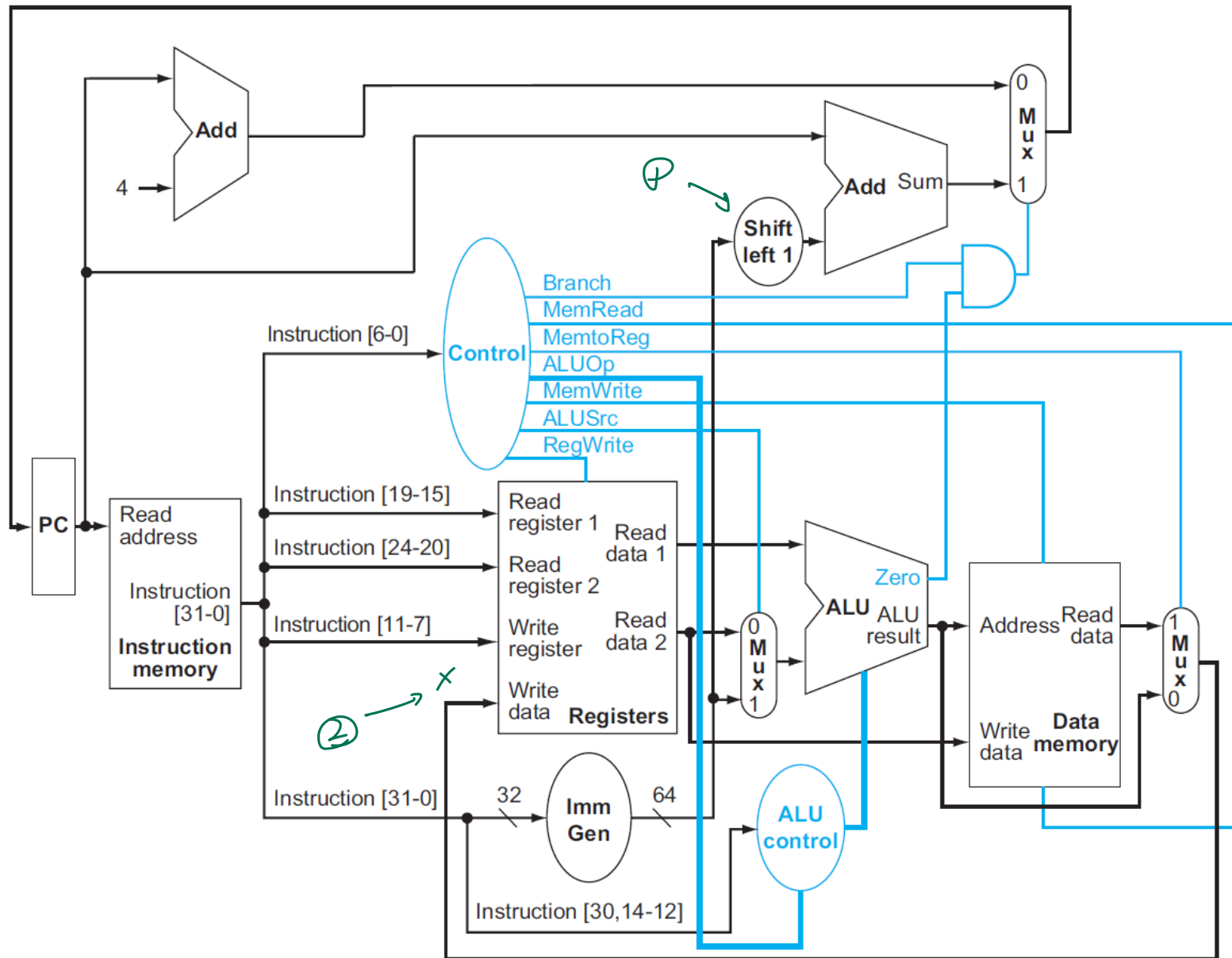
Upper bits sign-extended from inst[31] always

Format	Instruction Class	Notation	Effect
R-type	register-register ALU operations	op rd, rs1, rs2 (e.g., add)	$RF[rd] = RF[rs1] \text{ op } RF[rs2]$
I-type	register-immediate ALU operations	op rd, rs1, imm (e.g., addi)	$RF[rd] = RF[rs1] \text{ op } \text{sign_extend}(imm)$
	loads	lsize rd, imm(rs1) (sizes: lb, lh, lw, ld) (also: signed or unsigned)	addr = $RF[rs1] + \text{sign_extend}(imm)$ $RF[rd] = \text{MEM}[addr]$
	register	jr rs1	$PC = RF[rs1]$
	register-immediate	jalr rd, rs1, imm	$RF[rd] = PC + 4;$ $PC = RF[rs1] + \text{sign_extend}(imm)$ $PC = PC \& 0xffffffffe$ (set LSB 0)
S-type	stores	ssize imm(rs1), rs2 (sizes: sb, sh, sw, sd)	addr = $RF[rs1] + \text{sign_extend}(imm)$ $\text{MEM}[addr] = RF[rs2]$
B-type	conditional branches	bop rs1, rs2, imm (e.g., beq, bne, blt, etc.)	condition = $(RF[rs1] \text{ op } RF[rs2])$ target = $PC + \text{sign_extend}(imm) * 2$ $PC = (\text{condition} ? \text{target} : PC + 4)$
U-type	register-immediate	op rd, imm	$RF[rd] = imm \ll 12$ (auipc need add PC) → special inst
J-type	register-immediate	jal rd, imm	$RF[rd] = PC + 4;$ $PC = PC + \text{sign_extend}(imm) * 2$

Single-Cycle MIPS Implementation



Single-Cycle RISC-V Implementation



- End of Moore's Law and Dennard Scaling require innovation in Computer Architecture
 - Massive heterogeneous computing era, power, performance, area (PPA) becomes key SoC integration factor

- RISC-V instruction set architecture
 - Avoids "over-architecting" while allows efficient implementation
 - Small base ISA
 - Optional standard subset ISA extension
 - Custom instruction (accelerators)
 - Enabling agile hardware development
 - Open and common ISA (c.f. licensable or closed ISA)
 - Viable software development platform

New Golden Age for Domain Specific Architecture



在未來的世界裡，每個系統都有其特有之專屬化之技能（網路下載之版權圖片，請妥善使用）

Reference

- <https://tclin914.github.io/77838749/>
- <https://gist.github.com/internetsadboy/28d6862db63ecbbb2324>
- <https://www.slideshare.net/YiHsiuHsu/riscv-introduction>
- <https://en.wikipedia.org/wiki/RISC-V#Significance>
- <https://www.edntaiwan.com/news/article/20181211NT31-analysis-the-wave-of-RISC-V-processor>